tidySTL: Exposing Implicit Semantics in STL Evaluation

Sota Sato[1][0000-0001-7147-3989]✉ and Masaki Waga[2][0000-0001-9360-7490]

[1] AIST, Tokyo, Japan
sota.sato@aist.go.jp
[2] Kyoto University, Kyoto, Japan

Abstract. Robust semantics of Signal Temporal Logic (STL) provides a verdict, optimization objective, and training feedback across monitoring, falsification, and control. Evaluating a discretely sampled trace, however, requires decisions that neither the STL formula nor the samples fully specify, such as interpolation and end-of-trace behavior. We call these evaluator-level choices *implicit semantics*; they can vary across tools, altering robustness values and downstream results. We present tidySTL, a diagnostic toolkit that lets users evaluate the same formula and sampled trace under named behaviors of existing STL tools. By aligning intermediate robustness values along the shared formula structure, tidySTL localizes where two tool behaviors first diverge. We demonstrate this localization with compatibility backends for existing STL tools. The same architecture also serves as an extensible evaluator layer: new evaluation algorithms or semantics require only modular backend changes, while keeping the formula, input trace, and public interface fixed.

1 Introduction

Signal Temporal Logic (STL) is a standard formalism for specifying temporal properties of real-valued signals [9,7]. Its quantitative *robust* semantics measures how strongly a behavior satisfies or violates a specification, and the resulting margin underpins runtime monitoring [2], falsification [1], and learning-enabled design [12].

The robust semantics is defined over an idealized continuous signal, yet tools usually evaluate it on finite, discretely sampled traces, including finite prefixes accumulated during runtime monitoring. This requires evaluator-level choices that are not determined by the STL formula and samples alone, such as interpolation and end-of-trace behavior. We call these choices *implicit semantics*; Table 1 organizes the main sources of divergence.

Because these choices are often embedded inside each tool, two STL tools can report different robustness values or Boolean satisfaction verdicts for the same formula and trace. Such discrepancies have appeared in practice: validation in the ARCH-COMP 2021 falsification-tool competition had to account for different robustness encodings of discrete gear predicates, alongside other validation mismatches [6]. Addressing such cases requires identifying the evaluator

choice responsible for a discrepancy, locating where its effect first appears in the formula, and tracing how that local difference propagates to the reported result (Section 4.2).

We propose tidySTL,[3] a diagnostic toolkit for localizing such discrepancies. Rather than comparing final outputs alone, tidySTL evaluates the same formula and trace under named tool behaviors, aligns their intermediate robustness values along a shared formula structure, and reports where they first diverge (Figure 1). This finite-trace workflow can diagnose recorded prefixes from runtime monitoring, traces produced during falsification, and simulation or test logs.

This paper makes the following contributions.

- We organize the implicit semantic choices in STL robustness evaluation and show that they can affect robustness values and Boolean satisfaction verdicts (Section 2 and Table 2).
- We propose tidySTL, with a common interface and representation for STL evaluator behavior, allowing both existing tools and new evaluator variants to share a single formula-and-trace frontend (Section 3 and Figure 1).
- We turn cross-tool discrepancies into diagnosable evaluation differences by localizing where two evaluator behaviors first diverge. We demonstrate this with compatibility backends for Breach [4], TaLiRo [1], and RTAMT [11] (Section 4.2 and Table 4).

Related Work. Most STL tools follow the classical robustness of Donzé and Maler [5] or Fainekos and Pappas [7]. Classical implementations include Breach for verification and parameter synthesis [4], S-TaLiRo for falsification [1], and RTAMT for online monitoring [11]; tidySTL compares evaluator-level choices across these uses. Beyond classical robustness, some tools further implement extended semantics or logics: STLCG++ [8] for smooth robustness, mstlo [14] for RoSI-style finite-prefix uncertainty [3], CauMon [15] for causation, and MoonLight [10] for spatio-temporal properties. Supporting a broader range of these semantics is future work.

2 Implicit Semantic Choices in STL Evaluation

We take the quantitative STL semantics of Donzé and Maler [5] and the closely related framework of Fainekos and Pappas [7] as our theoretical reference. This setting underlies most existing STL monitoring and falsification tools [6]. We focus on its evaluator-level variations and recall only the definitions needed to delimit them; the cited works give the full semantics.

Definition 1 (Signal). *A* signal *over a set of variables* $\mathbf{Var}$ *is a function* $\sigma\colon \mathbb{R}_{\geq 0} \to \mathbb{R}^{\mathbf{Var}}$. *For* $\mathtt{x} \in \mathbf{Var}$, *we write* $\sigma_{\mathtt{x}}(t)$ *for the* $\mathtt{x}$*-component of* $\sigma(t)$.

[3] Artifact repository: `https://github.com/midoriao/tidystl`.

Table 1. Representative *implicit choices* in implementing STL robustness evaluation. These choices can affect both reported robustness values and the accepted class of formula-trace inputs. Further formal details are given in the extended version [13].

Axis	Choice
Finite-trace interpretation	Reading samples as a signal, affects robustness
Signal model	Piecewise-linear | Piecewise-constant | Discrete
Signal beyond horizon	Clamp | Pessimistic
Robustness at horizon	Exact | Copy-prev
Predicate semantics	Atomic-level semantics and supported forms
Distance metric	Signed margin | Euclidean (affects robustness)
Nonstandard predicates	E.g., Equality (affects acceptance/robustness)
Logic fragment	Which parts of the full logic are supported, affects acceptance
Unbounded intervals	Unbounded operators like $\Box_{[0,\infty)}$ supported?
Until/release	$\mathcal{U}_I$ and $\mathcal{R}_I$ supported?
Open intervals	Bounds such as (a, b) and $(a, b]$ supported?
Top/bottom	$\top(+\infty)$ and $\bot(-\infty)$ supported?
Real-valued bounds	Non-integer temporal bounds supported?
Implementation-specific	Conventions affecting robustness or acceptance
Off-grid windows	Exact endpoints | Samples only | Empty-window fallback
Non-uniform time	As-is | Force equal spacing | Reject
Infinity encoding	IEEE $\pm\infty$ | Finite sentinels

Definition 2 (STL syntax). *STL formulas are generated by the following grammar:*

$$\varphi ::= \top \mid f(\boldsymbol{x}) \geq 0 \mid \neg\varphi \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \,\mathcal{U}_I\, \varphi_2 \mid \varphi_1 \,\mathcal{R}_I\, \varphi_2 \mid \Diamond_I \varphi \mid \Box_I \varphi.$$

Here f is a function symbol, for example $f(\mathtt{x}, \mathtt{y}) = 4\mathtt{x} - \mathtt{y} - 3$, and I is a non-singular interval in $\mathbb{R}_{\geq 0}$, for example $[2, 5]$, $(1, 3]$, or $[0, \infty)$.

Informally, the robust semantics $\rho(\varphi, \sigma, t)$ replaces the Boolean verdict with a signed real-value, e.g., $\rho(\mathtt{x} - 3 \geq 0, \sigma, t) = \sigma_{\mathtt{x}}(t) - 3$. The Boolean connectives and temporal operators then aggregate the robustness values using min, max, inf, and sup; for example, $\rho(\Box_{[0,2]}\,(\mathtt{x} - 3 \geq 0), \sigma, t) = \inf_{t' \in [t,\, t+2]}(\sigma_{\mathtt{x}}(t') - 3)$.

We call a design choice in a concrete evaluator's implementation an *implicit semantic choice* if the choice can change the resulting robustness evaluation even for the same formula and trace. For example, an evaluator often receives only a finite *timed state sequence* $(t_0, \boldsymbol{v}_0), \ldots, (t_n, \boldsymbol{v}_n)$ of timestamped samples, whereas the reference semantics is defined over an idealized signal. How this finite representation is interpolated into a signal over $\mathbb{R}_{\geq 0}$ is an implicit semantic choice. Table 1 organizes the choices we consider.

Example 1 (Verdict flip from one implicit choice). Consider the *signal beyond horizon* choice of Table 1. Let $\varphi = \Diamond_{[2,4]}(\mathtt{x} \geq 0)$ be evaluated at $t = 0$ on a

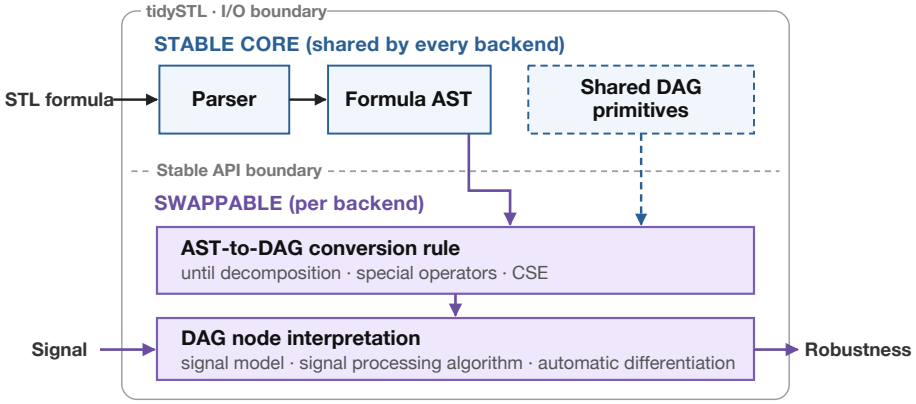


Fig. 1. Architecture of tidySTL: a stable core shared by every backend and a swappable backend layer (lowering and node interpretation) that isolates the implicit semantic choices.

trace ending at $T = 1$, whose final sample assigns 3 to `x`. The temporal window $[2, 4]$ lies entirely beyond the trace. A clamping rule treats the last sample as persisting and reports $+3$ (satisfied), while a pessimistic rule treats the window as unavailable and reports $-\infty$ (violated). Thus, the same formula and trace can receive an opposite Boolean verdict depending on this choice.

Neither rule is intrinsically wrong: a finite trace underdetermines the idealized signal. Rather than declaring one value correct, a discrepancy can therefore be diagnosed by localizing the implicit choice that produced it. In Section 4.2, we will apply this principle to discrepancies between tool behaviors on concrete formula-and-trace inputs.

3 Design and Implementation

This section turns the implicit semantic choices of Section 2 into inspectable implementation structure. tidySTL exposes robustness evaluation through a uniform interface, while placing choices that vary across evaluators behind a backend boundary.

3.1 Evaluation Interface

tidySTL has two explicit inputs: formula and signal. A *formula* is a typed syntax tree, parsed from the textual STL frontend. A *signal* contains one or more named, batched value sequences over a shared time grid.

An optional third input, the *backend*, encapsulates the implicit semantic choices and the execution strategy. The call `robustness(formula, signal, backend)` returns an array of robustness values over time:

```python
from tidystl import Signal, parse, robustness
import numpy as np

phi = parse("G[0,5](F[0,2](velocity >= 10))")

times = np.linspace(0, 10, 200)
velocity_batch = np.array([np.sin(times), np.cos(times)])  # two traces
sig = Signal.from_dict(
    times=times,
    values={"velocity": velocity_batch},
)
rho = robustness(phi, sig)                    # default backend
rho = robustness(phi, sig, backend="breach")  # Breach-compatible
```

Keeping this interface small serves two purposes. For users, the same formula and signal can be evaluated under different documented backends without changing the surrounding application. For backend authors, new evaluation behavior can be implemented behind a stable interface rather than requiring a new parser, signal representation, or calling convention.

3.2 Architecture

Internally, tidySTL separates a stable core from backend-specific evaluation logic, as shown in Figure 1. The stable core is shared by every backend, and it does not decide how (φ, σ) maps to a robustness value. Those decisions are delegated to the backend layer.

This separation makes intermediate evaluation *observable*. Because two backends build their outputs on the same shared formula (syntax-tree) representation, those outputs can be aligned at corresponding formula nodes. A post-order walk can therefore identify minimal divergent nodes, whose primitive operations and intermediate signals remain available for inspection. Section 4.2 demonstrates this diagnostic workflow.

Each backend lowers the formula AST into a backend-specific computation DAG (directed acyclic graph), and then interprets its typed primitive operations as a concrete signal-processing pipeline. This keeps signal-processing factors separate from the syntactic and recursive handling of STL formulas. Figure 2 illustrates such a lowering on an example formula with nested temporal operators.

Extension points. A backend defines how the formula is lowered into a computation DAG and how the resulting DAG nodes are interpreted. Extensions may introduce new node types during lowering or override how existing nodes are interpreted. The artifact repository includes the user manual (`docs/usage.md`) and reference implementations. In Section 4.1 we show that emulating an existing tool requires only compact backend-specific changes.

Architectural byproducts. Beyond this paper's scope of cross-tool discrepancy diagnosis, the swappable architecture (Figure 1) also enables new evaluation

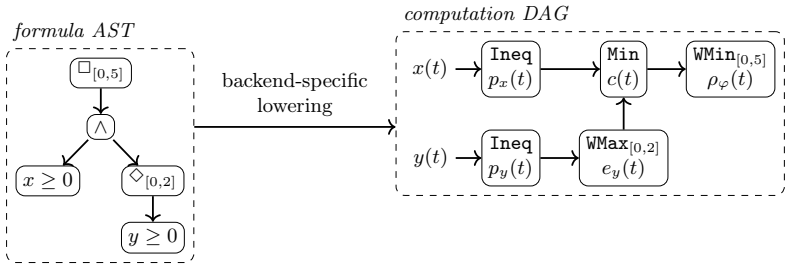


Fig. 2. Lowering of $\varphi :\equiv \square_{[0,5]}\big((\mathtt{x} \geq 0) \wedge \diamond_{[0,2]}(\mathtt{y} \geq 0)\big)$ from AST (left) to computation DAG (right). Each DAG node is a primitive operation: the conjunction reduces pointwise while the temporal operators aggregate over their annotated window. Edges carry intermediate robustness signals that remain inspectable in the evaluation result.

strategies to be implemented through the same interface. The codebase ships a differentiable (smooth robustness [8]) backend and an SIMD-optimized backend as instances of this extensibility.

4 Empirical Study

We evaluate tidySTL 0.2.0 in two parts.[4] First, we assess backend modularity, compatibility with reference-tool outputs, and runtime performance (Section 4.1). Second, using the validated backends, we localize cross-tool discrepancies to a formula node and expose the operations involved (Section 4.2).

Our evaluation uses fixed versions of Breach 1.11.4, RTAMT 0.3.5 (discrete default and dense, RTAMT$_{den}$), and TaLiRo (SVN revision 146). Each diagnostic pairs a formula and trace chosen so that candidate behaviors produce distinct robustness values at the marked readout. Matching the observed output to these predictions identifies a tool's behavior along one implicit-choice axis without inspecting its implementation. Table 2 compares the observed behaviors along three axes.

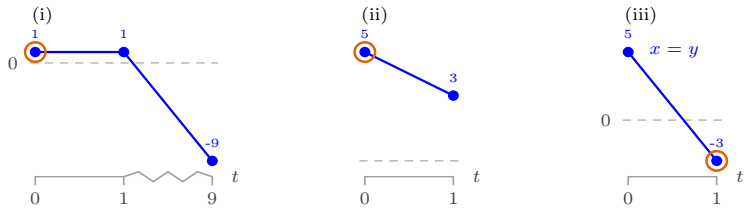
Tool groupings vary across the diagnostics: RTAMT$_{den}$ aligns with different tools on different axes. Thus, no pairwise correspondence captures all observed behaviors.

4.1 Backend Modularity, Compatibility, and Performance

We implemented compatibility backends for Breach, TaLiRo, and RTAMT to reproduce their observed evaluator behavior within tidySTL. The resulting variant-specific implementations each comprise fewer than 200 lines: 85 for Breach, 167 for TaLiRo, and 193 for RTAMT, excluding blank and comment lines.

[4] The artifact repository includes the implementation used in our empirical study, together with scripts and documentation (`extra/experiments/`).

Table 2. Diagnostic comparison of Breach, TaLiRo, and RTAMT in discrete and dense modes. Each row isolates one finite-trace choice. *Top*: sampled traces used by diagnostics (i)–(iii); horizontal positions follow sample index, tick labels show actual timestamps, and the orange ring marks the reported sample. *Bottom*: the behavior inferred from each tool's output, with the observed robustness in parentheses. *n/o*: the choice is not observable from TaLiRo's whole-trace scalar output.



Implicit choice (test formula & signal)	Breach	TaLiRo	RTAMT	RTAMT$_{den}$
Signal model	dense	dense	discrete	dense
`G[0,2](x>=0)` (i)	(1)	(1)	(−9)	(1)
Signal beyond horizon	clamp	pessimistic	pessimistic	clamp
`F[2,4](x>=0)` (ii)	(3)	(−∞)	(−∞)	(3)
Robustness at horizon	copy-prev	*n/o*	exact	exact
`(x>=0) and (y>=0)` (iii)	(5)		(−3)	(−3)

For Breach, the common DAG representation already suffices, so only node-interpretation overrides are needed. TaLiRo and RTAMT instead require custom lowering to represent tool-specific operations with dedicated DAG nodes (e.g., normalizing TaLiRo predicates by the coefficient-vector norm and converting RTAMT temporal intervals to discrete sample offsets), as well as custom node interpretation. All changes remain within the backend boundary, leaving the stable core and frontend unchanged.

Compatibility validation. We validate each compatibility backend against outputs from the corresponding reference tool on a suite covering its operators. These outputs are generated in the tools' native environments and stored as committed reference fixtures. Reproduction tests evaluate the same formulas and traces with the backend and compare robustness at every reported timestep for Breach and RTAMT, and at $t = 0$ for TaLiRo, whose interface returns only a whole-trace scalar.

Performance. We benchmark $\square_{[0,5]}((\mathtt{x} > 0) \wedge \Diamond_{[0,2]}(\mathtt{y} > 0))$ on synthetic signals sampled at equally spaced points over $[0, 10]$. Table 3 reports single-trace and batched results for different time-grid sizes. The results show that tidySTL's inspectable, swappable backend design retains practical performance: its compatibility backends are competitive with RTAMT and STLCG++ on single traces, while vectorization remains effective across batches.

Table 3. Mean evaluation time in ms (10 runs) for $N = 1$ ($N = 256$); "-c." denotes a tidySTL compatibility backend. RTAMT batches use a Python loop; STLCG++ uses native `torch.vmap`.

T	Breach-c.	TaLiRo-c.	RTAMT-c.	RTAMT	STLCG++
101	0.5 (1.1)	0.5 (1.1)	0.3 (0.8)	0.3 (49.8)	0.2 (4.8)
1001	5.3 (25.8)	4.5 (24.9)	2.2 (22.3)	2.4 (485.1)	3.8 (556.2)

Table 4. Pairwise localization using validated compatibility backends. Root robustness values (ρ) follow the tool order; the final column gives their root-level effect and the minimal divergent node. Formulas appear in Examples 2 and 3; `window_sampling` is shown in Figure 3.

Case	Tools compared	$\lvert\varphi\rvert$/depth	ρ	Divergence
`until_boundary`	Breach vs. RTAMT	3 / 1	-5 / $+2$	verdict flip on $\mathcal{U}_{[1,2]}$
`window_sampling`	Breach vs. TaLiRo	10 / 3	$+2$ / $+5$	value gap on $\square_{[1,2]}$

4.2 Localizing Cross-Tool Discrepancies

The diagnostic cases in Table 2 reveal differences along isolated implicit-choice axes, but a concrete formula may exercise only a subset or interaction of those choices. We therefore use the validated compatibility backends to identify which behavior causes an observed mismatch and where it first affects the evaluation. Table 4 summarizes the results for two concrete formula-and-trace inputs.

For each backend pair, the localizer compares aligned per-node traces on the shared syntax tree. A single post-order walk reports a *minimal divergent node*—a lowest node whose backend outputs disagree while all of its children still agree—and its first divergent timestep. Backend pairs whose lowerings differ structurally are compared at the shared syntax-tree level; predicate-internal arithmetic is treated as a leaf. Localization identifies the first divergence between these backend implementations.

Example 2 (Off-grid window localization). Consider the following CPS-style requirement:

$$\square_{[0,3]}\Big((\texttt{mode} \geq 0) \wedge \Diamond_{[0,1]}\big((\texttt{enable} \geq 0) \wedge \square_{[1,2]}(\texttt{level} \geq 0)\big)\Big) \wedge (\texttt{safety\_margin} \geq 0).$$

The formula combines a nested temporal response with the independent constraint $\texttt{safety\_margin} \geq 0$. The trace has eight samples: $t = 0, 3$, then unit-spaced through 9; `level` is $3, 0$, then 5. Thus $[1, 2]$ contains no sample. The outer $\square_{[0,3]}$ requires the system to remain in the required mode and reach an enabled point within one time unit. From that point, `level` must remain nonnegative at offsets $[1, 2]$. On the gap from $t = 0$ to $t = 3$, the innermost $\square_{[1,2]}$ has no sampled point to reduce over. As Figure 3 shows, this is the first node where Breach and TaLiRo differ, three temporal levels below the root; both Boolean verdicts remain satisfied, so final verdict comparison would miss it.

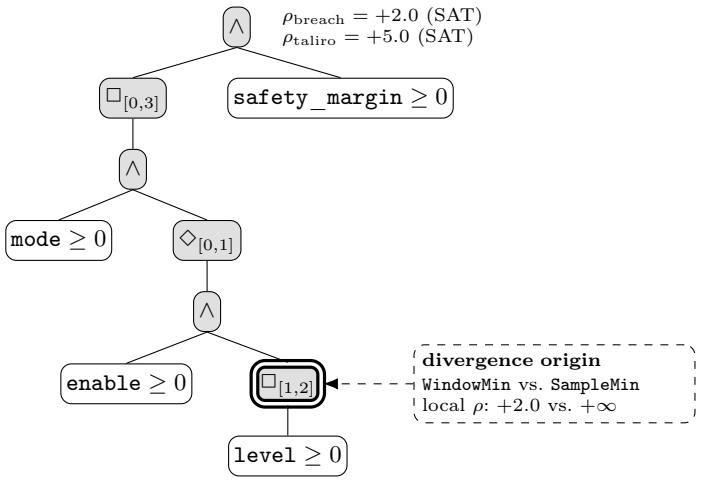


Fig. 3. Localization of the Breach–TaLiRo divergence in Example 2. At $t = 0$, the highlighted $\square_{[1,2]}$ window spans $[1, 2]$, entirely between the samples at $t = 0$ and $t = 3$. Breach interpolates $\texttt{level}(1) = 2$, whereas TaLiRo finds no sample and returns $+\infty$. Shading traces this first disagreement to the roots: Breach's $+2.0$ remains binding, whereas TaLiRo's vacuous branch leaves $+5.0$. Both tools satisfy the formula despite reporting different robustness.

Example 3 (Until verdict flip). For $(\texttt{x} \geq 0)\, \mathcal{U}_{[1,2]}\, (\texttt{y} \geq 0)$ in `until_boundary`, the two samples are at $t = 0, 1$ with $\texttt{x} = [5, -5]$ and $\texttt{y} = [-1, 2]$, so $[1, 2]$ reaches past the trace. Breach and TaLiRo report -5.0, while RTAMT reports $+2.0$. The localizer pins the discrepancy to the single $\mathcal{U}_{[1,2]}$ node.

The same pattern applies to the ARCH-COMP case discussed in Section 1. Attributing the reported mismatch between the tools to their predicate encodings (± 1 versus $\pm\infty$) required manual debugging. tidySTL replaces that task with automatic node-wise localization, exposing the responsible evaluator behavior as an explicit, comparable algorithmic difference.

Taken together, Examples 2 and 3 demonstrate the two complementary outcomes of localization. The former reveals a robustness discrepancy hidden by agreeing Boolean verdicts; the latter identifies the operation responsible for a verdict flip. These results show that node-wise comparison explains cross-tool discrepancies both when Boolean verdicts agree and when they differ.

5 Conclusion and Future Work

tidySTL exposes implicit choices at the STL evaluator boundary. Separating formula syntax, signal data, backend lowering, and DAG interpretation pinpoints the operations where evaluations first diverge. Future work will broaden the supported semantics, including online evaluation, and expand the validation suite.

References

1. Annpureddy, Y., Liu, C., Fainekos, G.E., Sankaranarayanan, S.: S-TaLiRo: A tool for temporal logic falsification for hybrid systems. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, vol. 6605, pp. 254–257. Springer (2011). https://doi.org/10.1007/978-3-642-19835-9_21
2. Bartocci, E., Deshmukh, J., Donzé, A., Fainekos, G., Maler, O., Ničković, D., Sankaranarayanan, S.: Specification-based monitoring of cyber-physical systems: A survey on theory, tools and applications. In: Lectures on Runtime Verification: Introductory and Advanced Topics, Lecture Notes in Computer Science, vol. 10457, pp. 135–175. Springer (2018). https://doi.org/10.1007/978-3-319-75632-5_5
3. Deshmukh, J.V., Donzé, A., Ghosh, S., Jin, X., Juniwal, G., Seshia, S.A.: Robust online monitoring of signal temporal logic. Formal Methods in System Design **51**(1), 5–30 (2017). https://doi.org/10.1007/s10703-017-0286-7
4. Donzé, A.: Breach, a toolbox for verification and parameter synthesis of hybrid systems. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, vol. 6174, pp. 167–170. Springer (2010). https://doi.org/10.1007/978-3-642-14295-6_17
5. Donzé, A., Maler, O.: Robust satisfaction of temporal logic over real-valued signals. In: Formal Modeling and Analysis of Timed Systems (FORMATS). Lecture Notes in Computer Science, vol. 6246, pp. 92–106. Springer (2010). https://doi.org/10.1007/978-3-642-15297-9_9
6. Ernst, G., Arcaini, P., Bennani, I., Chandratre, A., Donzé, A., Fainekos, G., Frehse, G., Gaaloul, K., Inoue, J., Khandait, T., Mathesen, L., Menghi, C., Pedrielli, G., Pouzet, M., Waga, M., Yaghoubi, S., Yamagata, Y., Zhang, Z.: ARCH-COMP 2021 category report: Falsification with validation of results. In: 8th International Workshop on Applied Verification of Continuous and Hybrid Systems (ARCH21). EPiC Series in Computing, vol. 80, pp. 133–152. EasyChair (2021). https://doi.org/10.29007/xwl1
7. Fainekos, G.E., Pappas, G.J.: Robustness of temporal logic specifications for continuous-time signals. Theoretical Computer Science **410**(42), 4262–4291 (2009). https://doi.org/10.1016/j.tcs.2009.06.021
8. Kapoor, P., Mizuta, K., Kang, E., Leung, K.: STLCG++: A masking approach for differentiable signal temporal logic specification. IEEE Robotics and Automation Letters **10**(9), 9240–9247 (2025). https://doi.org/10.1109/LRA.2025.3588389
9. Maler, O., Nickovic, D.: Monitoring temporal properties of continuous signals. In: Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT). Lecture Notes in Computer Science, vol. 3253, pp. 152–166. Springer (2004). https://doi.org/10.1007/978-3-540-30206-3_12
10. Nenzi, L., Bartocci, E., Bortolussi, L., Silvetti, S., Loreti, M.: MoonLight: A lightweight tool for monitoring spatio-temporal properties. International Journal on Software Tools for Technology Transfer **25**(4), 503–517 (2023). https://doi.org/10.1007/s10009-023-00710-5
11. Nickovic, D., Yamaguchi, T.: RTAMT: Online robustness monitors from STL. In: Automated Technology for Verification and Analysis (ATVA). Lecture Notes in Computer Science, vol. 12302, pp. 564–571. Springer (2020). https://doi.org/10.1007/978-3-030-59152-6_34
12. Raman, V., Donzé, A., Maasoumy, M., Murray, R.M., Sangiovanni-Vincentelli, A., Seshia, S.A.: Model predictive control with signal temporal logic specifications. In:

53rd IEEE Conference on Decision and Control (CDC). pp. 81–87. IEEE (2014). https://doi.org/10.1109/CDC.2014.7039363
13. Sato, S., Waga, M.: tidySTL: Exposing implicit semantics in STL evaluation (extended version). arXiv preprint (2026)
14. Thomsen, A.K., Madsen, N.V.S., Evans, V.T., Wright, T.D., Esterle, L., Larsen, P.G.: mstlo: Efficient online monitoring of signal temporal logic. arXiv preprint arXiv:2605.26847 (2026)
15. Zhang, Z., An, J., Arcaini, P., Hasuo, I.: Caumon: A tool for online monitoring against signal temporal logic. Sci. Comput. Program. **253**, 103499 (2026). https://doi.org/10.1016/J.SCICO.2026.103499